

JS FUNKCIJE

DEFINICIJA FUNKCIJE

- U JavaScript-u pojam *definicija funkcije* obuhvata deklaraciju funkcije, funkcionalne izraze i *arrow* funkcije.
- Sve ovo su načini da u kodu specificirate da je nešto funkcija.
- Deklaracija funkcije je samostalni izraz koji eksplicitno imenuje funkciju i definiše šta ona radi, a koristi ključnu reč *function*

DEKLARACIJA FUNKCIJE

```
function nazivFunkcije(parametar) {  
    // telo funkcije  
}
```

- Iza deklaracije funkcije se obično ne stavlja ; jer se separator koristi za odvajanje izvršivih izraza.
- Deklarisana funkcija se ne izvršava odmah - mora se pozvati negde iz ostatka koda npr:

```
// x prima rezultat izvršavanja funkcije  
let x = nazivFunkcije( "tekst za parametar" );
```

FUNKCIONALNI IZRAZ

- Funkcija se može definisati i putem izraza
(*function expression*)

```
const x = function (a, b) {return a + b};
```

- Kako ovde nema naziva funkcije u definiciji - kako se ova funkcija posle poziva?
 - Naziv varijable služi kao naziv funkcije.

```
let y = x(10, 27);
```

FUNKCIONALNI IZRAZ

- Često se na ovaj način definiše funkcija umesto putem deklaracije
- Varijabla praktično može biti *tipa* funkcije.
A funkcija za JS nije ništa drugo nego objekat.

```
const mojaFunkcija = function(a, b){return a * b};
```

```
let x = mojaFunkcija(4, 3);
```

FUNCTION HOISTING (PROMOCIJA FUNKCIJA)

- JavaScript dozvoljava da se varijabla, pa tako i funkcija deklariše **nakon** što je već negde upotrebljena. Ova osobina se na engleskom naziva *hoisting* (uzdizanje, promocija)
- *Hoisting* je JavaScript podrazumevano ponašanje - automatski pri interpretiranju koda “podiže” sve deklaracije na početak trenutnog opsega važenja (početak bloka, funkcije ili script fajla)
- Ovo podizanje na vrh opsega važi SAMO za deklaracije ne i za izraze inicijalizacije!!!

Kao posledica ovoga funkcije definisane izrazima se ne “promovišu”.

FUNCTION HOISTING (PRIMER)

Ovo može:

```
var y = saberi(10,18);  
  
function saberi(a,b){  
    return a+b;  
}
```

Ovo ne može:

```
var y = saberi(10,18);  
  
var saberi=function(a,b){return a+b;};
```

ARROW FUNKCIJE

- Koncizna sintaksa pisanja funkcija
- Osim što se funkcija može pridružiti promenljivoj sada je moguće i skratiti zapis same definicije funkcije (može delovati malo nečitko na prvi pogled)

- ranije:

```
let hello = function() {  
    return "Hello World!";  
}
```

- arrow notacija:

```
let hello = () => {  
    return "Hello World!";  
}
```

Ali ako funkcija ima samo return izraz može i kraće:

```
let hello = () => "Hello World!";
```

ARROW FUNKCIJE

- Kao i svaka funkcija *arrow* može imati ulazne parametre.

- ranije:

```
const add = function(a, b) {  
  return a+b;  
};
```

- arrow notacija:

```
const add = (a, b) => a+b;
```

- Ako funkcija ima samo jedan parametar, onda se mogu izostaviti i zagrade.

- ranije:

```
const square = function(num) {  
  return num*num;  
}
```

- arrow notacija:

```
const square = num => num*num;
```

ARROW FUNKCIJE

- Omogućava skraćivanje pisanja *callback* funkcija bez potrebe pisanja *function* ključne reči

- ranije:

```
const nums=[11,15,17,20,25,27,30,31];  
const out=[];  
  
nums.forEach(function(el) {  
    if (el % 5 === 0) out.push(el);  
});
```

- arrow notacija:

```
const nums=[11,15,17,20,25,27,30,31];  
const out=[];  
  
nums.forEach(el => {  
    if (el % 5 === 0) out.push(el)  
});
```

SAMOPOZIVAJUĆE FUNKCIJE

- Funkcionalni izraz se može napisati tako da funkcija bude samopozivajuća, tj. da odmah nakon definicije sledi njen poziv.
- Ovo je moguće samo kada se za definiciju funkcije koristi izraz. Nije moguće kada se koristi deklaracija funkcije.
- Ovo se često može videti u kodu koji koristi neki od JS Frameworka ili biblioteka (jQuery, Vue, ...)

SAMOPOZIVAJUĆE FUNKCIJE (PRIMER)

```
(function () {  
    var x = "Hello!!";  
    alert(x);  
})();
```

```
(function(message) {  
    alert(message);  
})('Moja poruka');
```

PARAMETRI FUNKCIJE

- Funkcije mogu, ali ne moraju imati parametre
- *Parametri* su nazivi zadati u definiciji funkcije
 - U JavaScript-u se ne definiše tip parametra
 - JavaScript ne radi proveru tipa prosleđenih argumenata
 - JavaScript funkcije NE proveravaju broj primljenih argumenata niti da li se on slaže sa brojem parametara navedenim u definiciji
- *Argumenti* su stvarne vrednosti koje se u momentu pozivanja funkcije prosleđuju funkciji.
- Argumenti se prosleđuju **po vrednosti**.
- Zbog načina kako JS radi sa objektima, **ako je argument objekat** on se prosleđuje **po referenci**.

PODRAZUMEVANA VREDNOST PARAMETARA

- Ukoliko se funkcija pozove sa nedovoljnim brojem argumenata (manjim od broja parametara) sve nedostajuće vrednosti će biti postavljene na specijalnu vrednost *undefined*
- Ovo je ponekad OK, ali je često dobro proveriti u kodu

```
function mojaFunkcija(a, b) {  
  if (b === undefined) {  
    b = 0;  
  }  
  return a*b;  
}
```

- Od verzije ES6 (2015. god) dozvoljeno je zadati podrazumevanu vrednost parametra pri deklaraciji funkcije

```
function mojaFunkcija(a=1, b=0) {  
  return a*b;  
}
```

„PREOSTALI PARAMETRI“

- „rest“ pattern
- Omogućava se agregacija preostalih argumenata pri pozivu funkcije u jedan imenovani parametar

// a je parametar - niz koji će primiti sve argumente

// pri pozivu funkcije posle prva dva

```
let myFunction = (x, y, ...a) {  
  return (x + y) * a.length  
}
```

// funkcija pozvana sa 5 parametara

// prva dva idu u x i y, a ostala 3 u niz a

```
myFunction(1, 2, "hello", true, 7) === 9 // true, jer je (1+2)*3
```